

Diffusion Policy · arXiv 2303.04137

Empiricist

11+3 · seat 4

T_a grid · DDIM · closed vs open loop

Sep 21 paper club · VLA Architectures

Spine: chunk-commit-replan (T_p / T_a) — NOT classical planner

Chi et al. · RSS 2023 / IJRR extended

Four hypotheses · Empiricist seat 4

H-horizon

$T_a \approx 8$ balances consistency
vs reactivity
(~ 0.8 s commit @ 10 Hz)

H-DDIM

Quality saturates
 ~ 10 – 16 steps;
extra buys latency only

H-pos

Position > velocity
for DP; opposite for
LSTM-GMM control

H-loop

Open-loop long chunks
fail under obs shift
(= VLA chunk failure)

Priority ladder

P0 T_a grid + closed vs open · P1 DDIM steps + pos/vel

P2 CNN vs Transformer · warm-start · Defer vision FT / Transport / full 15

Scout $\leq \sim 50$ GPU-h · Push-T state first ~ 4 – 12 GPU-h · env_outline/ ready

H-horizon · $T_a \in \{1, 2, 4, 8, 16\}$

Fix $T_o=2$ · $T_p=\max(16, T_a)$ · Push-T / Lift state first

$T_a = 1$

mode flicker
idle trap

too reactive

$T_a = 2$

still jittery
weak commit

under-smooth

$T_a = 4$

improving
consistency

rising

$T_a = 8$

SWEET SPOT
paper default

peak expected
TARGET

$T_a = 16$

open-loopish
slow recovery

sluggish

Empiricist P0

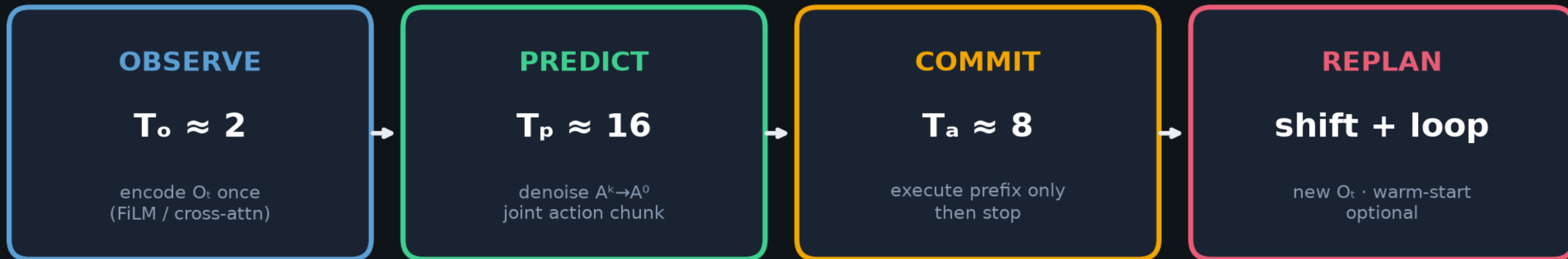
Metric: success / IoU · 1 seed · ~500-1500 ep · CNN · state

Hypothesis: peak near $T_a=8$; $T_a=1$ jitter; large T_a slow recovery

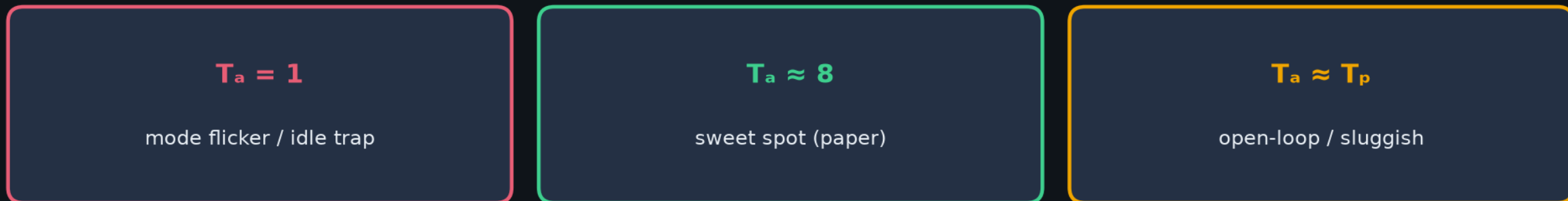
Budget: Push-T state ~4-12 GPU-h · scout $\leq$ ~50 GPU-h total

Chunk-Commit-Replan · the motion/control contract

NOT classical planner / Diffuser full-trajectory planning · actions-only, O-conditioned



One control cycle (@ ~10 Hz cmds → 125 Hz interp)



H-loop · closed-loop replan vs open-loop chunks

Same failure mode as long VLA chunks without replan

CLOSED-LOOP

$T_a < T_p$ · replan every commit

1. $O_t \rightarrow$ denoise chunk

2. exec T_a prefix

3. new O_{t+ta}

4. replan (warm-start)

OPEN-LOOP

$T_a \approx T_p$ · rare / no mid replan

1. $O_t \rightarrow$ denoise long chunk

2. exec almost all of T_p

3. obs may have shifted

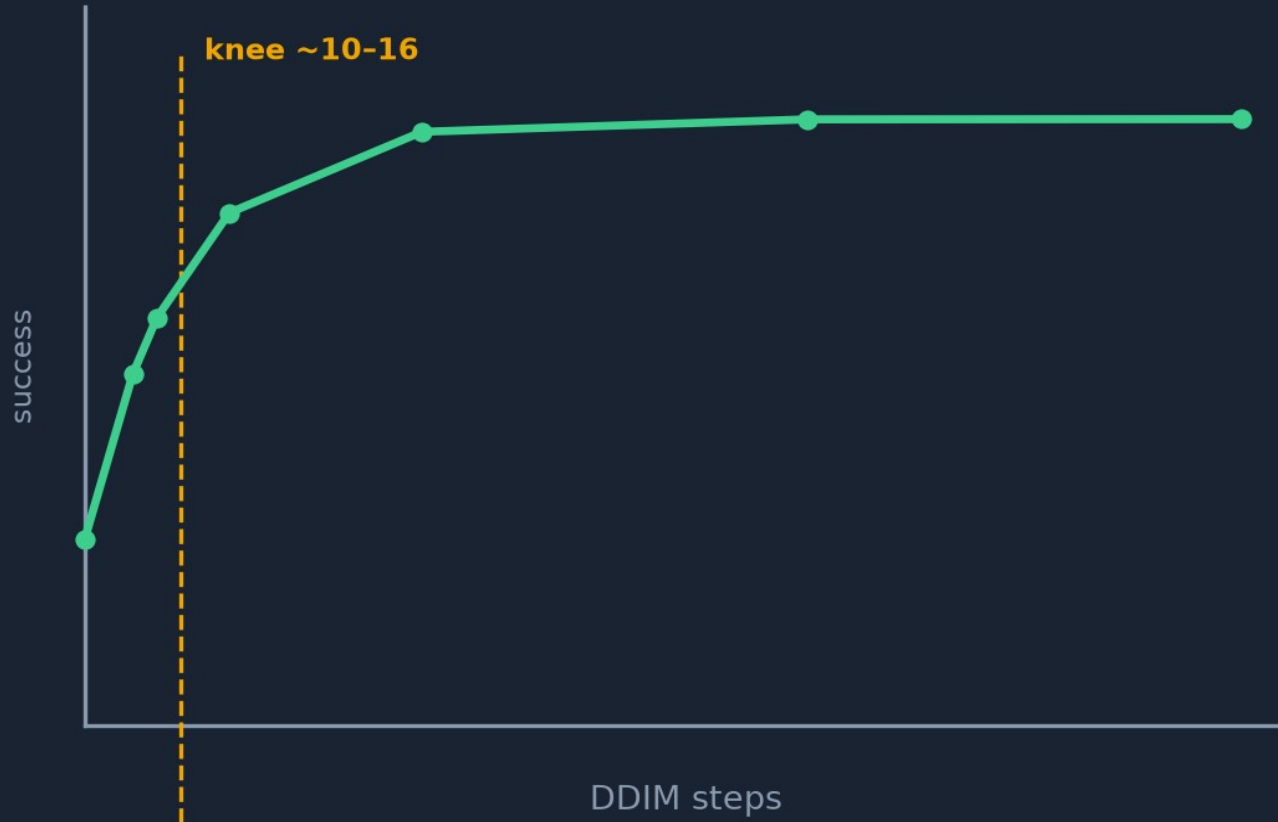
4. recovery fails / lags

Empiricist test: set $T_a=T_p$ and disable mid-episode replan; inject obs delay 0-4 steps

Expect clear gap at delay ≥ 2 — ancestor of VLA long-chunk brittleness

H-DDIM · quality saturates ~10-16 steps

Train 100 (iDDPM square-cosine) → infer DDIM 10-16 for real-time



Real-time budget

~0.1 s @ 10-16 steps

RTX 3080 (paper)

cmds 10 Hz → 125 Hz

P1 experiment

steps {4..100}
infer-only on ckpt

< 1 GPU-h

H-pos · position > velocity for Diffusion Policy

Baselines often prefer velocity — shared action interface is the fair cite

POSITION chunks

- Multimodality = which geometric path
 - Less compounding error over T_p
 - Idle / dwell actions natural
- Latency-robust (~ 4 steps, Fig 5)

VELOCITY chunks

- Integrates error open-loop
- CNN oversmooths sharp cmds
 - Idle = zero-vel ambiguity
- Older BC stacks preferred this

Reviewer hygiene: DP@position vs baselines@velocity inflates headline Δ

Empiricist P1: rerun Lift/Square PH state with shared action alphabet

Expect: position $\Delta > 0$ for DP; opposite direction for LSTM-GMM control



Figure 3. Multimodal behavior. At the given state, the end-effector (blue) can either go left or right to push the block. **Diffusion Policy** learns both modes and commits to only one mode within each rollout. In contrast, both **LSTM-GMM** [Mandekar et al. \(2021\)](#) and **IBC** [Florence et al. \(2021\)](#) are biased toward one mode, while **BET** [Shafiiullah et al. \(2022\)](#) fails to commit to a single mode due to its lack of temporal action consistency. Actions generated by rolling out 40 steps for the best-performing checkpoint.

speculate that there are two primary reasons for this discrepancy: First, action multimodality is more pronounced in position-control mode than it is when using velocity control. Because Diffusion Policy better expresses action multimodality than existing approaches, we speculate that it is inherently less affected by this drawback than existing methods. Furthermore, position control suffers less than velocity control from compounding error effects and is thus more suitable for action-sequence prediction (as discussed in the following section). As a result, Diffusion Policy is both less affected by the primary drawbacks of position control and is better able to exploit position control’s advantages.

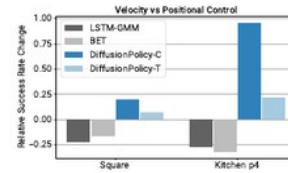


Figure 4. Velocity v.s. Position Control. The performance difference when switching from velocity to position control. While both BCRNN and BET performance decrease, Diffusion Policy is able to leverage the advantage of position and improve its performance.

4.3 Benefits of Action-Sequence Prediction

Sequence prediction is often avoided in most policy learning methods due to the difficulties in effectively sampling from high-dimensional output spaces. For example, IBC would struggle in effectively sampling high-dimensional action space with a non-smooth energy landscape. Similarly, BC-RNN and BET would have difficulty specifying the number of modes that exist in the action distribution (needed for GMM or k-means steps).

In contrast, DDPM scales well with output dimensions without sacrificing the expressiveness of the model, as demonstrated in many image generation applications. Leveraging this capability, Diffusion Policy represents action in the form of a high-dimensional action sequence, which naturally addresses the following issues:

- **Temporal action consistency:** Take Fig 3 as an example. To push the T block into the target from the bottom, the policy can go around the T block from either left or right. However, suppose each action in the sequence is predicted as independent multimodal distributions (as done in BC-RNN and BET). In that case, consecutive actions could be drawn from different modes, resulting in jittery actions that alternate between the two valid trajectories.
- **Robustness to idle actions:** Idle actions occur when a demonstration is paused and results in sequences of identical positional actions or near-zero velocity actions. It is common during teleoperation and is sometimes required for tasks like liquid pouring. However, single-step policies can easily overfit to this pausing behavior. For example, BC-RNN and IBC often get stuck in real-world experiments when the idle actions are not explicitly removed from training.

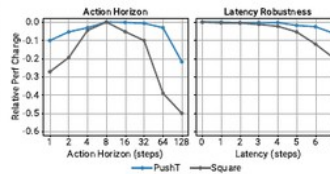


Figure 5. Diffusion Policy Ablation Study. Change (difference) in success rate relative to the maximum for each task is shown on the Y-axis. **Left:** trade-off between temporal consistency and responsiveness when selecting the action horizon. **Right:** Diffusion Policy with position control is robust against latency. Latency is defined as the number of steps between the last frame of observations to the first action that can be executed.

4.4 Training Stability

While IBC, in theory, should possess similar advantages as diffusion policies. However, achieving reliable and high-performance results from IBC in practice is challenging due to IBC’s inherent training instability [Ta et al. \(2022\)](#). Fig 6 shows training error spikes and unstable evaluation performance throughout the training process, making hyperparameter tuning critical and checkpoint selection difficult. As a result, [Florence et al. \(2021\)](#) evaluate every checkpoint and report results for the best-performing checkpoint. In a real-world setting, this workflow necessitates the evaluation of many policies on hardware to select a final policy. Here, we discuss why Diffusion Policy appears significantly more stable to train.

An implicit policy represents the action distribution using an Energy-Based Model (EBM):

$$p_{\theta}(\mathbf{a}|\mathbf{o}) = \frac{e^{-E_{\theta}(\mathbf{o}, \mathbf{a})}}{Z(\mathbf{o}, \theta)} \quad (6)$$

where $Z(\mathbf{o}, \theta)$ is an intractable normalization constant (with respect to $\mathbf{a}$).

GPU budget scout

Quiet plan — outlines only; no cluster jobs launched

SETUP	VRAM	TIME	PRI
Push-T / Lift state CNN	8-12 GB	4-12 h	P0 start
Square image CNN	16-24 GB	4-10 h scout	P3 defer
DDIM sweep infer-only	same ckpt	< 1 h	P1 cheap
Pos vs vel Lift/Square	8-12 GB	8-20 h	P1
Full 15-task × 3 seed	—	100s GPU-h	DO NOT

Pass bar: reproduce qualitative paper trends (T_a peak, DDIM knee, closed>open) — not +46.9%

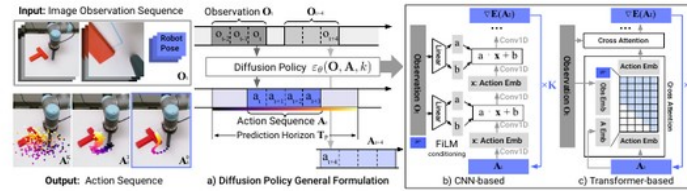


Figure 2. Diffusion Policy Overview a) General formulation. At time step t , the policy takes the latest T_o steps of observation data O_t as input and outputs T_a steps of actions A_t . b) In the CNN-based Diffusion Policy, FILM (Feature-wise Linear Modulation) [Perez et al. \(2018\)](#) conditioning of the observation feature O_t is applied to every convolution layer, channel-wise. Starting from A_t^k drawn from Gaussian noise, the output of noise-prediction network ϵ_θ is subtracted, repeating K times to get A_t^k , the denoised action sequence. c) In the Transformer-based [Vaswani et al. \(2017\)](#) Diffusion Policy, the embedding of observation O_t is passed into a multi-head cross-attention layer of each transformer decoder block. Each action embedding is constrained to only attend to itself and previous action embeddings (causal attention) using the attention mask illustrated.

2.2 DDPM Training

The training process starts by randomly drawing unmodified examples, x^0 , from the dataset. For each sample, we randomly select a denoising iteration k and then sample a random noise ϵ^k with appropriate variance for iteration k . The noise prediction network is asked to predict the noise from the data sample with noise added.

$$\mathcal{L} = \text{MSE}(\epsilon^k, \epsilon_\theta(x^0 + \epsilon^k, k)) \quad (3)$$

As shown in [Ho et al. \(2020\)](#), minimizing the loss function in Eq 3 also minimizes the variational lower bound of the KL-divergence between the data distribution $p(x^0)$ and the distribution of samples drawn from the DDPM $q(x^0)$ using Eq 1.

2.3 Diffusion for Visuomotor Policy Learning

While DDPMs are typically used for image generation (x is an image), we use a DDPM to learn robot visuomotor policies. This requires two major modifications in the formulation: 1. changing the output x to represent robot actions. 2. making the denoising processes *conditioned* on input observation O_t . The following paragraphs discuss each of the modifications, and Fig. 2 shows an overview.

Closed-loop action-sequence prediction: An effective action formulation should encourage temporal consistency and smoothness in long-horizon planning while allowing prompt reactions to unexpected observations. To accomplish this goal, we commit to the action-sequence prediction produced by a diffusion model for a fixed duration before replanning. Concretely, at time step t the policy takes the latest T_o steps of observation data O_t as input and predicts T_p steps of actions, of which T_a steps of actions are executed on the robot without re-planning. Here, we define T_o as the observation horizon, T_p as the action prediction horizon and T_a as the action execution horizon. This encourages temporal action consistency while remaining responsive. More details about the effects of T_o are discussed in Sec 4.3. Our formulation also allows receding horizon control [Mayne and Michalska \(1988\)](#) to further improve action smoothness by warm-starting the next inference setup with previous action sequence prediction.

Visual observation conditioning: We use a DDPM to approximate the conditional distribution $p(A_t|O_t)$ instead of the joint distribution $p(A_t, O_t)$ used in [Janner et al. \(2022a\)](#) for planning. This formulation allows the model to predict actions conditioned on observations without the cost of inferring future states, speeding up the diffusion process and improving the accuracy of generated actions. To capture the conditional distribution $p(A_t|O_t)$, we modify Eq 1 to:

$$A_t^{k-1} = \alpha(A_t^k - \gamma \epsilon_\theta(O_t, A_t^k, k) + \mathcal{N}(0, \sigma^2 t)) \quad (4)$$

The training loss is modified from Eq 3 to:

$$\mathcal{L} = \text{MSE}(\epsilon^k, \epsilon_\theta(O_t, A_t^0 + \epsilon^k, k)) \quad (5)$$

The exclusion of observation features O_t from the output of the denoising process significantly improves inference speed and better accommodates real-time control. It also helps to make **end-to-end** training of the vision encoder feasible. Details about the visual encoder are described in Sec. 3.2.

3 Key Design Decisions

In this section, we describe key design decisions for Diffusion Policy as well as its concrete implementation of ϵ_θ with neural network architectures.

3.1 Network Architecture Options

The first design decision is the choice of neural network architectures for ϵ_θ . In this work, we examine two common network architecture types, convolutional neural networks (CNNs) [Ronneberger et al. \(2015\)](#) and Transformers [Vaswani et al. \(2017\)](#), and compare their performance and training characteristics. Note that the choice of noise prediction network ϵ_θ is independent of visual encoders, which will be described in Sec. 3.2.

CNN-based Diffusion Policy We adopt the 1D temporal CNN from [Janner et al. \(2022b\)](#) with a few modifications: First, we only model the conditional distribution $p(A_t|O_t)$ by conditioning the action generation process on observation features O_t with Feature-wise Linear Modulation (FILM)

Fig 2 · horizons $T_o/T_p/T_a$ + CNN/Transformer

arXiv 2303.04137

Q & A

T_a · DDIM · closed loop — scout first

Diffusion Policy · Sep 21

Diffusion Policy: Visuomotor Policy Learning via Action Diffusion

Cheng Chi^{*1}, Zhenjia Xu^{*1}, Siyuan Feng², Eric Cousineau², Yilun Du³, Benjamin Burchfiel², Russ Tedrake^{2,3}, Shuran Song^{1,4}

Abstract

This paper introduces Diffusion Policy, a new way of generating robot behavior by representing a robot's visuomotor policy as a conditional denoising diffusion process. We benchmark Diffusion Policy across 15 different tasks from 4 different robot manipulation benchmarks and find that it consistently outperforms existing state-of-the-art robot learning methods with an average improvement of 46.9%. Diffusion Policy learns the gradient of the action-distribution score function and iteratively optimizes with respect to this gradient field during inference via a series of stochastic Langevin dynamics steps. We find that the diffusion formulation yields powerful advantages when used for robot policies, including gracefully handling multimodal action distributions, being suitable for high-dimensional action spaces, and exhibiting impressive training stability. To fully unlock the potential of diffusion models for visuomotor policy learning on physical robots, this paper presents a set of key technical contributions including the incorporation of receding horizon control, visual conditioning, and the time-series diffusion transformer. We hope this work will help motivate a new generation of policy learning techniques that are able to leverage the powerful generative modeling capabilities of diffusion models. Code, data, and training details is available diffusion-policy.cs.columbia.edu

Keywords

Imitation learning, visuomotor policy, manipulation

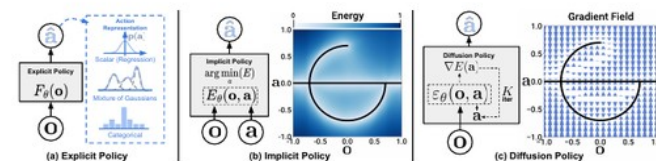


Figure 1. Policy Representations. a) Explicit policy with different types of action representations. b) Implicit policy learns an energy function conditioned on both actions and observation and optimizes for actions that minimize the energy landscape c) Diffusion policy refines noise into actions via a learned gradient field. This formulation provides stable training, allows the learned policy to accurately model multimodal action distributions, and accommodates high-dimensional action sequences.

1 Introduction

Policy learning from demonstration, in its simplest form, can be formulated as the supervised regression task of learning to map observations to actions. In practice however, the unique nature of predicting robot actions — such as the existence of multimodal distributions, sequential correlation, and the requirement of high precision — makes this task distinct and challenging compared to other supervised learning problems.

Prior work attempts to address this challenge by exploring different *action representations* (Fig 1 a) — using mixtures of Gaussians [Mandlekar et al. \(2021\)](#), categorical representations of quantized actions [Shafiq et al. \(2022\)](#), or by switching the *policy representation* (Fig 1 b) — from explicit to implicit to better capture multi-modal distributions [Florence et al. \(2021\)](#); [Wu et al. \(2020\)](#).

In this work, we seek to address this challenge by introducing a new form of robot visuomotor policy that

generates behavior via a “conditional denoising diffusion process” [Ho et al. \(2020\)](#) on robot action space”, **Diffusion Policy**. In this formulation, instead of directly outputting an action, the policy infers the action-score gradient, conditioned on visual observations, for K denoising iterations (Fig. 1 c). This formulation allows robot policies to inherit several key properties from diffusion models — significantly improving performance.

- **Expressing multimodal action distributions.** By learning the gradient of the action score function [Song and Ermon \(2019\)](#) and performing Stochastic Langevin Dynamics sampling on this gradient field, Diffusion policy can express arbitrary normalizable distributions [Neal et al. \(2011\)](#), which includes multimodal action distributions, a well-known challenge for policy learning.